

Operators in Project Reactor

In Project Reactor, operators are the functional building blocks used to create, transform, filter, combine, and orchestrate asynchronous data streams (handled via Flux for 0-N items, or Mono for 0-1 items). They allow you to define complex reactive assembly lines while managing backpressure without blocking.



Each operator returns a new Flux or Mono. They never modify the original stream.

Creation Operators

These operators initiate the reactive stream by creating publishers from values, collections, or time-based sources.

1. `just()` – Emits a specific, predefined set of values. Creates a **cold** publisher — data is generated only when someone subscribes.

```
Flux<String> fruits = Flux.just("apple", "banana", "cherry");
```

2. `fromIterable()` / `fromStream()` – Converts Java collections or streams into a Flux. Most commonly used to convert database query results (List) into a reactive stream.

```
Flux<String> fromList = Flux.fromIterable(List.of("A", "B", "C"));
```

3. `range()` – Creates a sequence of integers. Internally very efficient; used heavily for pagination and batch processing.

```
Flux<Integer> numbers = Flux.range(1, 5); // 1,2,3,4,5
```

4. `interval()` – Emits sequential numbers at fixed time intervals (great for polling/tickers).

```
Flux<Long> ticks = Flux.interval(Duration.ofSeconds(1));  
// Emits 0,1,2,3... every second
```

5. `fromCallable()` – Wraps blocking or lazy computations. The standard way to wrap blocking legacy code (JDBC, RESTTemplate, etc.) into reactive world.

```
Mono<String> result = Mono.fromCallable(() -> slowDatabaseCall());
```

Data Transformation Operators

We'll use this base stream for consistency:

```
Flux<String> fruits = Flux.just("apple", "banana", "cherry",  
"date", "eggs");
```

6. `map()` – Transforming Data. Maintains **1:1** relationship and is the fastest transformation operator.

```
fruits.map(String::toUpperCase)  
      .subscribe(System.out::println);
```

Output: APPLE, BANANA, CHERRY, DATE, eggs

7. `flatMap()` – Flattening Asynchronous Streams. Used when each item produces a new reactive stream. Flattens nested publishers. Can change the order of results because inner streams complete at different speeds (very common interview question).

```
fruits.flatMap(fruit -> Flux.just(fruit + "1", fruit + "2"))  
      .subscribe(System.out::println);
```

Filtering, Limiting & Aggregating Operators

8. `filter()` – Filtering Data Streams. The condition is checked only when downstream requests data.

```
fruits.filter(fruit -> fruit.length() > 5)
      .subscribe(System.out::println);
```

Output: banana, cherry, eggs

9. `take()` & `skip()` – Limiting the stream. Useful for pagination in reactive repositories.

```
fruits.take(3).subscribe(System.out::println);    // apple,
banana, cherry
fruits.skip(2).subscribe(System.out::println);    // cherry,
date, eggs
```

10. `count()` – Count total items. Operator waits for the stream to complete (`onComplete`) before emitting the final count.

```
fruits.count()
      .subscribe(count -> System.out.println("Total: " + count));
```

Output: Total: 5

11. `sort()` – Sorting elements. Buffers all elements in memory before sorting — be careful with large datasets.

```
fruits.sort(Comparator.reverseOrder())
      .subscribe(System.out::println);
```

Output: eggs, date, cherry, banana, apple

12. `reduce()` – Aggregating values. Similar to `Stream.reduce()` but reactive; great for aggregations like sum, max, average.

```
Flux<Integer> numbers = Flux.just(1, 2, 3, 4, 5);
numbers.reduce(0, Integer::sum)
    .subscribe(sum -> System.out.println("Sum = " + sum));
```

Output: Sum = 15

13. `groupBy()` – Grouping elements

```
fruits.groupBy(fruit -> fruit.length())
    .flatMap(group -> group.collectList()
        .map(list -> group.key() + " -> " + list))
    .subscribe(System.out::println);
```

Combining & Shaping Streams

14. `concat()` / `concatWith()` – Combining Sequentially. Guarantees order by waiting for first stream to complete before starting next.

```
Flux<String> moreFruits = Flux.just("fig", "grape");
Flux.concat(fruits, moreFruits)
    .subscribe(System.out::println);
```

15. `merge()` / `mergeWith()` – Combining Concurrently. Does **not** guarantee order but is faster as streams run in parallel.

```
Flux.merge(fruits, moreFruits)
    .subscribe(System.out::println);
```

16. `zip()` – Pairing Elements. Stops when the **shortest** stream ends — very useful when joining multiple service calls.

```
Flux<String> colors = Flux.just("red", "yellow", "black", "brown", "purple");
```

```
Flux.zip(fruits, colors, (f, c) -> f + " is " + c)
    .subscribe(System.out::println);
```

17. `collectList()` – Most common way to convert `Flux<T> → Mono<List<T>>` for REST API responses.

```
fruits.collectList()
    .subscribe(list -> System.out.println(list));
```

Mono Specific Operators

18. `single()` – Enforce exactly one item. Throws exception if more than one item is emitted — good for strict validation.

```
Mono.just("Hello").single()
    .subscribe(System.out::println);
```

19. `block()` – Blocking (Only for testing)

```
String value = Mono.just("Hello").block(); // blocks the calling thread here
```

Warning: Never use in production WebFlux code.

20. `toFuture()` – Convert to `CompletableFuture`. Helps in bridging reactive code with traditional async code (`CompletableFuture`).

```
Mono.just("Future Data")
    .toFuture()
    .thenAccept(System.out::println);
```

21. `repeat()` – Repeat emission

```
Mono.just("Hello")
    .repeat(3)
```

```
.subscribe(System.out::println);
```

Output: Hello Hello Hello Hello

22. `then()` – Ignores previous result and continues — commonly used for fire-and-forget operations after save.

```
Mono.just("Process this")
    .doOnNext(data -> System.out.println("Processing: " + data))
    .then(Mono.just("Final Result"))
    .subscribe(System.out::println);
```

Output: Final Result